



The Software Quality Iceberg

The term 'software quality' continues to be an oxymoron! In this article, we explore the meaning of the term to understand why we need to focus on internal quality to develop high quality software.

Within just a few decades, software has come to pervade our lives. It is surprising that people don't realise how intricately connected with software their lives are. There is considerable software in the car you drive, the microwave oven with which you cook, the TV you watch, the mobile you use, the planes in which you fly, the projectors you use for making presentations... the list is endless. Even if you know which devices need software, you may be surprised by the extent to which they rely on it. Do you know that high-end cars these days have over a million lines of code? Or were you aware that the smart TVs that we watch nowadays and the planes in which you fly are aided by software that have over a million lines of code?

With the pervasive use of software, monitoring its quality has become more important than ever before. Most of you—even if you've nothing to do with computers—would have faced problems because of low-quality software. For example, if you visit banks often, you would certainly have experienced, first-hand, the problems with the banking software -- it takes time for boot-up and log in, will often crash, will not provide or allow certain features, etc. For these reasons, the banking staff members are forced to make you wait longer, or ask you to come back later. Recently, when I had to visit a branch of one of India's largest private-sector banks on a weekend, I found that the staff were asking all the customers to go home; the central server had crashed because of the huge volume of transactions, and no work could be taken up till the problem was resolved!

'Quality' itself is a complex concept, and since software is intangible, 'software quality' is even more complex. Let me illustrate why this is so and how 'quality' could be measured differently, depending on the viewpoint.

Based on whether you are a consumer of software, the software developer, or the organisation that develops the software, the meaning of high-quality software would differ. As a user, you would consider it high-quality if the application responds very fast, is very reliable, and easy to use, among other desirable characteristics. If you're developing the software, you'll rate software to be of high quality if you're able to easily make changes and test the software. Organisations that produce the software will find the software to be of high quality if

they are able to reuse it, port it and make it available on other platforms, or make it inter-operate with existing systems. As you can see, the same software can be perceived to be of high or low quality, depending on who you are.

Let me give you a specific example. I came across an enterprise software product that was widely deployed and used by customers worldwide. Since it evolved over time and matured, it was 'stable' in the sense that it was a product that customers could trust—it was efficient, reliable, and easy to use. The organisation that developed it was very happy with the product since it was a cash cow — hundreds of thousands of copies of the software product continued to sell worldwide. Further, the product was easily portable to different platforms. However, if you checked with the engineers who maintained the software, they were frustrated. The software was built on age-old technology nobody used anymore and, with two decades of changes, the software had accumulated so much code, that no one understood it any more. Further, the software was not designed with testing in mind, so making changes was a nightmare—any change in the code would often break the software. This symptom is known as 'code rot', and developers dread touching such software. Obviously, the software was reaching its retirement age, and was perceived as of 'low-quality' by the testing engineers. Yet, ironically, this same software was perceived as of high quality by the organisation that sold the software and by those who used it.

However, there is more to it. Gradually, over a period of a few years, the software had become so 'rigid' that no more changes could be made to it. Finally, the software product had to be scrapped, and rewritten from scratch! This led to the company losing billions. In other words, though on the surface, the different views on the software's quality seem independent of each other, they are intricately interconnected!

There are many other perspectives of software quality. For example, Capers Jones talks about the functional, aesthetic and structural aspects of software quality. As users, we care about the functionality that the software provides (i.e., functional software quality) and how easy it is to use (i.e., aesthetic software quality). However, as developers, we care about additional aspects of the software, such as its structural